ReqCheqAI Paper

Kondwani Mtawali & Grant Wells

Dr. Marcus Gubanyi

CS 331 Software Engineering

Introduction

In software development, creating well rounded, detailed software requirements is the key to creating long lasting software applications. Much like databases, without a proper design and specific boundaries, systems can be affected later in their life span. As we mentioned in class, poor requirement specifications can create delays, communication, and can have significant financial implications. We wanted to build an application that developers could use to analyze their requirements and clearly break down their requirement and score it based on the context that we have established. ReqCheq utilizes an large language model (LLM) to analyze the user's requirement, break it down into clear sections: functional, non-functional, ambiguity score, and active voice as noted in Figure 1.

```
OUTPUT: Structured JSON
    "Respond ONLY in JSON format as specified. Do not include explanations or extra text. Do not use code fences"
    {{
        "type:" "Functional or Non-Functional, or Functional + Non-Functional",
        "functional_portion": "Exact portion of the requirement describing system behavior",
        "non_functional_portion": "Exact portion describing constraints or quality attributes",
        "class:" "Functional, Performance, Security, Usability, Maintainability, Reliability, or N/A",
        "ambiguity_score": "0-100",
        "active_voice": "Yes or No"
    }}
```

Figure 1: What the LLM uses to structure the response to the user.

Not only does ReqCheck breakdown your requirement, but it also will return a score and feedback utilizing the context given to the LLM as seen in Figure 2 and Figure 3.

```
TASK1: Identify whether the requirement is a functional requirement, a non-functional requirement, or both.
    TASK1 DEFINITIONS:
    - Functional: Describes what the system should do, its behavior(the what)
    - Non-Functional: Describe the constraints under which the system should behave(the how)
    - Functional + Non-Functional: Describes both what the system should do and under what conditions the system must do it
    TASK1 CONSIDERATIONS: If the requirement includes both Functional and Non-Functional information, output Functional + Non-Functional
    TASK 1 OUTPUT: Functional || Non-Functional || Functional + Non-Functional

    TASK2: Identify the functional and non-functional portions of the given software requirement and output the separate portions
    TASK2 DEFINITIONS:
    - Functional: Describes what the system should do, its behavior
    - Non-Functional: Describe the constraints under which the system should behave
    TASK2 CONSIDERATIONS:
    - If either a functional or non-functional portion is not present, output N/A
    TASK2 OUTPUT: The exact of the functional and/or non-functional portion of the requirement

    TASK3: Identify the class the software requirement is being referred to in the requirement
    TASK3 DEFINITIONS:
    - Functional: A software requirement without any specifications of constraints or the conditions the feature must satisfy
    - Performance: Describes how efficiently the system operates under specific conditions, including speed, responsiveness, and resource usage
    - Security: Describes how the system protects data and prevents unauthorized access or misuse.
    - Usability: Describes how easy and intuitive the system is for users to interact with.
    - Maintainability: Describes how easily the system can be modified, updated, or extended over time.
    - Reliability: Describes the system's ability to operate consistently without failure over time.
    TASK3 CONSIDERATIONS:
    - A software requirement is capable of referencing multiple non-functional categories e.g., Performance and Security:
    - A software requirement may not have any non-functional components to it, resulting in being classified as Functional
    TASK3 OUTPUT: Output one, or multiple, classes the software requirement belongs to, or just "Functional" if there's no non-functional portion to the requirement

    TASK 4: Assign an ambiguity score to the requirement on a scale from 0 to 100.
    DEFINITION:
    - 0-33 (High Ambiguity): vague, subjective, multiple interpretations, not directly implementable
    - 34-66 (Moderate Ambiguity): partially clear but missing precision or detail
    - 67-100 (Low Ambiguity): clear, precise, measurable, directly implementable
```

Figure 2: The context given to the LLM on how to evaluate the user's requirement.

```
EVALUATION PROCESS (MANDATORY):
Evaluate the requirement using the following signals:

1. Vague Language:
Does the requirement include vague or subjective terms (e.g., "fast", "efficient", "user-friendly")?
→ Yes → increases ambiguity

2. Measurable Criteria:
Does the requirement include numbers, thresholds, or constraints?
→ Yes → reduces ambiguity

3. Clarity of Behavior:
Is the system behavior clearly defined and specific?
→ No → increases ambiguity

4. Implementability:
Could a developer implement this requirement without making assumptions?
→ No → increases ambiguity

SCORING INSTRUCTIONS:
- Start at 50 (moderate ambiguity)
- Add +10 to +20 for each factor that reduces ambiguity
- Subtract -10 to -20 for each factor that increases ambiguity
- Clamp final score between 0 and 100


TASK4: Classify whether or not the requirement is written in active voice structure
TASK4 DEFINITION:
- Requirements written in active voice follow this sequence: agent → verb → target
- Agent → The actor, the main subject
- Verb → the action being done
- Target → the subject being acted upon
TASK4 OUTPUT: Yes or No
```

Figure 3: Context continued..

The motivation for this project comes from the fact that, other than manually verifying, there are limited solutions to making sure software requirements are suitable for projects. It is also because the process of manually verifying is time consuming and much like testing, automating saves time. It is crucial to note that the vision for ReqCheq is to be a way to quickly check how well requirements are written and how well they specify what is needed. However, it should not be used as a one stop solution, it is designed to be like a pair programmer where you bounce ideas off of it. This software is worth building because it improves the end-user's experience. By allowing the user to write stronger requirements, it allows for less rework or refactoring, improve stakeholder and client communication and understanding, and it would allow for a more efficient development process. Picture this, a software developer gets some vague, poor requirements from a client, they would picture what is needed to be changed, they would put requirements into ReqCheq, and then they could take vague requirements and create well written and well structured software requirements.

Time Logs

Kondwani:

Date	*Time Spent*	*Work Completed*
3/30	2 hours	Project set up, understanding FastAPI, initializing app, writing first end point
3/31	1 hours	Additional set up and specifying project design prior to implementation
04/07	4 hours	Writing /analyze endpoint, prompt and response, API Key exposure, making the LLM call
04/12	4 hours	Test driven development for database functions, writing a method to compute an overall score, setting up the database following FastAPI DB docs
04/14	4 hours	Completing TDD cycle on database functions
04/25	10 hours	TDD cycle on score computations methods to add to the analyze endpoint, completing and adjusting DB functionality
04/26	6.5 hours	Small automated test on overall score function, new /feedback endpoint to provide qualitative analysis on requirement, wrote an react function to call the feedback endpoint and display LLM response to frontend, merged frontend changes

Grant:

Time Spent	Tasks	Work Completed
1 Hour 30 Minutes	Created a proof of concept for the frontend.	Yes
2 Hours	Building out a basic frontend utilizing React	Not completely, white web page is in place but title, input box, and button is not visible.
3 Hours	Building out the envisioned frontend with centered input box, button that changes upon a click.	No, working on finalizing the container and the style of it.
2 Hours 30 Minutes	Refactoring previous frontend build in order to future proof for final design.	Yes
3 Hours 45 Minutes	Beginning to style the frontend such as centering the input field, styling the input box adding a custom submit button.	No
1 Hour 30 Minutes	The center input box is centered now, but the submit button is off as well as the manual error check for invalid input isn't currently working. I also tested and validated the overflow feature and made sure that the scroll wheel only appears when the input is a certain size relative to the input field. Styling the input box by adding a colored border but working on the design of the actual input box.	Kind of
45 Minutes	Figured out the rounded edges for the input box	Yes
2 Hours	Fixing styling bugs. Fixed centering issues with the input box after the color change. Still working on styling the frontend with a proper, minimal header.	Yes & No
30 Minutes	About to link the frontend and backend so making sure that links between the two are correct.	Yes
4 Hours 30 Minutes	Working through errors and wiring the frontend, backend, and API altogether	Yes, core MVP functionality is now done. ReqCheq is working now.
2 Hours	Styling the response and working through related bugs.	Yes

Requirements

The original core functional and non-functional features included user input, an analyzer button to initiate request, a generated analysis report, functional and non-functional detection, ambiguity detection, and a computed overall requirement score.

The input requirement specified that the users of the application will need to submit string input for the system to run analysis on. The system will allow a user to input a single software requirement as text input, the system will validate the input, and the system will reject invalid inputs and display an error message. The requirement has been satisfied with a react frontend accepting user input, validating the input, and finally sending it to the backend for analysis and feedback processing.

The analyzer button requirement specified that the user must be able to click a button, triggering the analysis of the inputted requirement. This requirement has been satisfied using a react event handler that initiates a request to the backend upon clicking of the button.

The analysis report requirement specified that the system should generate an analysis report of the inputted requirement with the following sections:

- Functional and Non-Functional Requirements
- Ambiguity Detection
- Active Voice Formatting
- An Overall Requirement Score

The non-functional constraints included: the report will be generated within 10 seconds, each section of the analysis will be visually separated from the other sections, and the analysis report will generate without a page reload. The requirement has been nearly entirely satisfied, it does not satisfy the non-functional constraint of the report being generated within 10 seconds for all cases on an analysis request. In some cases, the system may take longer than 10 seconds to output the analysis report to the user. Besides that one non-functional restraint, all the conditions of this requirement have been satisfied.

In the backend, the system leverages an LLM call to OpenAI's gpt-4o-mini model(subject to change) with a prompt that specifies the following: the detection of functional and non-functional portions of the inputted requirements, an assigned ambiguity score out of 100, and the detection of Active Voice structure. The LLM is instructed to return its response in JSON format which is parsed by a method that computes an overall score based on the values assigned by the LLM.

In addition to the originally specified metrics of the analysis, the analysis report also includes the type of requirement it is(functional, non, or both), and the class of requirement it falls under. Additionally, more score computations than originally intended have been added: an atomicity score, measuring the verb count, a measurability score, counts the presence of numbers, complexity score, considers the number of clauses in the requirement, and a readability score, based on a well recognized reading ease function. These scores are computed utilizing sPacy, an NLP python library that provides utility in obtaining metrics from text data. The readability relies on a flesh_reading_ease function from textstat(python lib) that computes a readability score based on the average sentence length and average syllables per

word. All these new scores are factored into the computation of the overall score, along with the LLM JSON response.

Providing qualitative feedback is a core function of the web application that was not originally specified in the software requirements. Although not originally specified, providing qualitative feedback became a core feature which is a requirement that has been fully satisfied. It's been accomplished by saving the id of the analysis – reports are given id's in the DB – on the frontend when the /analyze end point returns the data to the frontend to output the necessary portions of the analysis. With that saved id, React makes calls to an asynchronous function to the backend to call the feedback endpoint. This endpoint consists of a prompt that accesses the requirement analysis report with the id of the report and makes another call to OpenAI's gpt-4o-mini model to provide feedback on the requirement, taking into consideration the report.

Design

Tech-Stack:

- FastAPI
- SQL Models
- React
- NPM
- Vite

Backend Design

The backend of the project is developed using FastAPI with an SQL Model Database. The frontend of the project is developed using React + Vite. The project is split up into two primary directories, a backend directory and a frontend. The backend directory contains the FastAPI application with a main.py file containing all the endpoints and logical structure of the application. The backend directory contains other key files including an environment variable storing the API key, tests.py which contains all the automated tests and TDD tests, models.py file which contains the schema for the database, and a database.py file where the CRUD database functions exists. A key design pattern used in our backend is in main.py, with the endpoint definitions. FastAPI uses decorators in defining the endpoints which are responsible for the HTTP request being made to the server. The class in the model acts as a builder design pattern as it facilitates the creation of multiple objects with the different attribute values, step by step the fields of the class are built uniquely and they're then constructed into an object stored in the database using the Requirement Analysis class.

Frontend Design

The frontend is developed in React whilst utilizing npm as the package/dependency manager and Vite as a build tool that provides a fast, responsive development environment. There's a few primary files unique to our platform in the frontend directory outside the necessary files that come with the React + Vite package. The main.jsx file acts as the entry point which initializes the application, the app.jsx file defines the app structure which is relatively simple in this web platform. The primary files edited include a ChatContainer.jsx file which defines the chat container, the form for the requirement submission by the user, and two asynchronous functions. The first async function is responsible for submitting the requirement to the backend to generate the analysis report and saving the id of the report for the feedback function.
The second async function is responsible for passing in the report id to the feedback endpoint on the backend.

Data Flow

The application follows a typical input-validation-output data flow as the primary object of the web application is to serve output to the UI based on input given by the user. When interacting with the system, the first step is inputting a user requirement(input string) into the react container that exists on the front end, when the analyze button is clicked, the submitHandle() async function is called which makes a POST request to the analyze endpoint on the backend, passing the inputted user string(the requirement) in the post request. The front end waits for the

analyze endpoint on the backend to send back the response for the sent POST request. The analyze endpoint on the backend receives the requirement and makes a call to OpenAI's gpt-4o-mini, the defined response the LLM must produce contains a prompt specifying how the LLM should analyze the requirement. The LLM is instructed to return the response in JSON format(an analysis report), the JSON formatted report is deserialized into a python dictionary, the four scores(atomicity, measurability, complexity, readability) are computed based on the requirement, and an overall score is computed based off the report and the four scores. The full analysis data is validated by Pydantic, and is sent to the frontend, along with the analysis id. The frontend parses the data, displaying the necessary information to the user. The analysis id originally stored is passed into an asynchronous function that calls the feedback endpoint on the backend. The feedback endpoint accesses the database to observe the requirement and its analysis information. A second LLM call is made to gpt-4o-mini, requesting that feedback is provided on the requirement. The endpoint generates a response in JSON formatted string, sends that JSON to the frontend, where react parses it and displays the necessary information to the user.

Implementation

Frontend Implementation

ReqCheq's frontend does much more than just look good by utilizing warm, inviting colors in order to attract attention from the user. Req Cheq's frontend is one of the primary points of how the user's requirements are analyzed and a report is given to the end-user. The following code snippet, from the ChatContainer function, is one of the key components of ReqCheq's frontend.

```
const [inputValue, setInputValue] = useState("")
const [response, setResponse] = useState("")
const [report, setReport] = useState("")
const [atomicScore, setAtomicScore] = useState("")
const [measureScore, setMeasureScore] = useState("")
const [complexScore, setComplexScore] = useState("")
const [readScore, setReadScore] = useState("")
const [reqScore, setReqScore] = useState("")
const [isLoading, setIsLoading] = useState(false)

// Feedback Endpoint State Variables
const [reqcheq_id, setreqcheq_id] = useState(null)
const [feedback, setFeedback] = useState(null)
const [isLoadingFeedback, setIsLoadingFeedback] = useState(false)

async function submitHandle() {
    // For DevTools + Manual Testing, shows the button is actually registering a button click when pressed.
    console.log("Button clicked")
    if (!inputValue.trim()) return
    setIsLoading(true)
    setResponse("")
    setReport("")
    setAtomicScore("")
    setMeasureScore("")
    setMeasureScore("")
    setComplexScore("")
    setReadScore("")
    setReqScore("")
    // Feedback Endpoint States
    setreqcheq_id(null)
    setFeedback(null)
```

Initialization & Form Submission

This piece of code above is from the ChatContainer function and it primarily utilizes React's useState hook to handle dynamic data within the interface. This is needed as when the user initially loads ReqCheq, there should be nothing within each category listed, however when generating the report, data fills these categories taking them from empty strings into dynamic

feedback for the user. The inputValue is capturing the user's typed requirement while isLoading handles the UI feedback during requests.

The analysis score states' five variables atomicScore, measureScore, complexScore, readScore, and reqScore all store the individual scoring context that is returned by the LLM. Response and report both hold the formatted text output of ReqCheq. The endpoint, reqcheq_id, stores the identifier returned after a successful response, which is used to associate the user feedback with a specific analysis. This avoids the error of having feedback being returned that doesn't relate to the user's current input requirement. The key part that allows this process to be set in motion is the submitHandle function. This is acting as the form's submission handler and guards against empty inputs with a trim check. In the original vision, the reports would be constantly stored and then displayed in a sidebar format, so, in a future proof development fashion, after the trim check, the values are reset before a request, which ensures that a prior input doesn't stay in the UI during the loading phase.

```
<div style={{ display: 'flex', justifyContent: 'flex-end' }}>
    <button onClick={submitHandle} disabled={isLoading}>
        {/* When reading disply: Analzing... + Button Name = Analyze*/}
        {isLoading ? "Analyzing..." : "Analyze"}
    </button>
</div>
{/* Display the response if there is one. */}
{/*  If there is no input = no response. */}
{report && (
    <div>
        <h2>Categorical / Quantitative Analysis</h2>
        <p>Type of Requirement: {report.type}</p>
        <p>Functional: {report.functional_portion}</p>
        <p>Non-Functional Portion: {report.non_functional_portion}</p>
        <p>Class: {report.class}</p>
        <p>Ambiguity Score: {report.ambiguity_score}</p>
        <p>Active Voice: {report.active_voice}</p>
        <p>Atomicity Score: {atomicScore}</p>
        <p>Measurability Score: {measureScore}</p>
        <p>Complexity Score: {complexScore}</p>
        <p>Readability Score: {readScore}</p>
        <p>Overall Requirement Score: {reqScore}</p>
    </div>
)}
{/* BUTTON: Generate Feedback */}
<button onClick={generateFeedback} disabled={isLoadingFeedback}>
    {/* When reading disply: Generating... + Button Name = GetFeedback*/}
    {isLoadingFeedback ? "Generating..." : "GetFeedback"}
</button>

{/* NEW FEEDBACK INFORMATION*/}
{feedback && (
    <div>
        <h2>Qualitative Feedback</h2>
        <h3>Disclaimer:</h3>
        <p>**{feedback.disclaimer}**</p>
        <h3>Feedback:</h3>
        <p>{feedback.feedback}</p>
        <h3>Requirement Strengths:</h3>
        <p>{feedback.requirement_strengths}</p>
        <h3>Requirement Weaknesses:</h3>
        <p>{feedback.requirement_weaknesses}</p>
        <h3>Suggestions:</h3>
        <p>{feedback.suggestions}</p>
        <h4>Improved Version</h4>
        <p>{feedback.improved_version}</p>
    </div>
```

Rendering & Results Display

This segment of code is for the handling of the analysis and displaying the results from a UI perspective. The analyze button is right-aligned using a flexbox style, however it is overwritten by the style formatting in the module.css file. The disabled={isLoading} statement is preventing multiple submissions while a request has already been submitted. For example, when looking at modern AI tools, once the AI begins responding to you, you can't submit another request to the model. This is essentially what the disabled={isLoading} attribute is handling. The label on the button is set to "Analyze", the text within the button before a submission is made. "Analyzing" is what is displayed within the button's format while the submission is being processed. This is a quality of life feature that gives the end-user some sort of feedback that allows them to see the system work in real time. It is similar to the text box border changing colors once you click into the input box. The {report && (...)} pattern is a conditional rendering operation. This is ensuring that the output is not displayed until the

successful completion of the API response. When rendered, it will display in the format as defined and specified by the backend.

Backend Implementation

The backend system is designed in FastAPI with an SQL model database, as mentioned prior. The main.py file contains all of the system logic. Primarily being composed of two endpoints. An endpoint is FastAPI's method for declaring URLs and the functionality that exists within it. Endpoints are configured with a decorator followed by the logic the endpoint must follow. Common practice in FastAPI development is validating the data types/structures the endpoint expects to receive, and in some cases also output.

Backend Implementation: Analyze Endpoint

The first endpoint written was the analyze endpoint. This endpoint is responsible for the core functionality of the web page. It includes a prompt, specifying instructions that an OpenAI's "gpt-4o-mini" model must create a response for. This prompt instructs LLM to complete the given tasks on the inputted requirement:

- Classify the requirement as functional, non-functional, or both functional + non-functional.
- Identify the functional and non functional components of the requirement
- Identify the class of non-functional requirement being inferred by the requirement
- Assigning an ambiguity score to the requirement on a scale of 0 to 100
- Determining whether the requirement is written in Active Voice structure or not

The LLM is instructed, and configured, to give a response in JSON formatted string consisting of the following fields:

```
OUTPUT: Structured JSON
"Respond ONLY in JSON format as specified. Do not include explanations or extra text. Do not use code fences"
{{
    "type:" "Functional or Non-Functional, or Functional + Non-Functional",
    "functional_portion": "Exact portion of the requirement describing system behavior",
    "non_functional_portion": "Exact portion describing constraints or quality attributes",
    "class:" "Functional, Performance, Security, Usability, Maintainability, Reliability, or N/A",
    "ambiguity_score": "0-100",
    "active_voice": "Yes or No"
}}


Requirement:
{request.requirement}
"""
# GREEN TEXT ABOVE = END OF PROMPT

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.0,
    response_format={"type": "json_object" }
)
```

The LLM returns its response in the form of JSON formatted string, as instructed, that JSON acts as the LLM "report", which is important to remember for when the full requirement analysis is saved to the database.

The next significant portion of the analysis endpoint is computing four scores, atomicity, measurability, complexity, and readability. These are all simple methods utilizing text analysis python libraries to derive insights from the inputted user requirement. The atomicity and measurability methods are given below:

```python
def atomicity_score(text: str):
    """
    Calculates an atomicity score. Good requirements describe one feature, counting verbs informs
    how many features the requirement is specifying for implementation.
    """
    doc = nlp(text)

    # token = smallest analytical unit processed in spacy(word,symbol,etc.)
    verbs = [token.lemma for token in doc if token.pos_ == "VERB"] # Counts all the verbs in the requirement

    # Assigning an atomicity score based on number of verbs
    if len(verbs) == 0:
        return 0
    elif len(verbs) <= 4:
        return 100
    elif len(verbs) == 5:
        return 50
    elif len(verbs) >= 6:
        return 25

def measurability_score(text: str):
    """
    Computes a measurability score by searching through the requirement and observing whether it contains
    a list of units, keywords, and/or spacy's like_num attribute. Spacy's like_num finds descriptions of
    numbers(.e.g. "hundreds", "ten", etc.). Measurable requirements are good requirements
    - like_nums get 40 points
    - units get 40 points
    - keywords get 20 points
    - returns a number that can't exceed 100
    """
    UNITS = {"seconds", "ms", "milliseconds", "%", "percent", "users", "requests"} # common unit terms we may see in requirements
    KEYWORDS = {"within", "at least", "at most", "no more than"} # keywords/phrases that imply measurement

    doc = nlp(text)

    score = 0

    # spacy like_num attribute
    if any(token.like_num for token in doc):
        score += 40

    if any(token.text.lower() in UNITS for token in doc):
        score += 40

    if any(keyword in text.lower() for keyword in KEYWORDS):
        score += 20

    return min(score, 100) # ensures score doesn't exceed 100, might fix later
```

Once all four scores have been computed. The overall score is computed by parsing the “report” generated by the LLM, identifying what the value of the fields are and attributing points based on those values, and also, attributing points to the results of the four(atomicity, measurability, complexity, readability) computed scores. A small portion of the overall score computation is given below:

```python
# Add a function to generate an overall score based on LLM generations
def overall_score(report: dict, atomicity: int, measurability: int, complexity: int, readability: int) -> int:
    """
    Computes the overall score of the requirement. Not necessary, more fun than anything else.
    """

    req_score = 0

    # Type Scoring
    req_type = report.get("type")
    match req_type:
        case "Functional + Non-Functional":
            req_score +=15
        case "Functional":
            req_score +=5
        case "Non-Functional":
            req_score +=5

    # Function Portion Scoring
    functional_portion = report.get("functional_portion")
    match functional_portion:
        case "N/A":
            req_score -= 10
        case _:
            req_score += 5

    # Non-Function Portion Scoring
    non_functional_portion = report.get("non_functional_portion")
    match non_functional_portion:
        case "N/A":
            req_score -= 10
        case _:
            req_score += 10

    # Class Scoring
    req_class = report.get("class")
    match req_class:
        case "Functional":
            req_score +=5
        case _:
```

Once the requirement has been assigned the appropriate scores, the full requirement analysis consisting of the scores, the report generated, and the requirement itself is saved to the database. Every requirement contains a unique id, which is included in the response of the endpoint.

```python
# The methods that compute scores are given the requirement
atomicity = atomicity_score(request.requirement)
measurability = measurability_score(request.requirement)
complexity = complexity_score(request.requirement)
readability = readability_score(request.requirement)

# The overall score is computed based on the report(python dict) & all the computed score
req_score = overall_score(report_dict, atomicity, measurability, complexity, readability)

"""
Database Save Portion:
- Define the reqcheq being saved
- Initialize the session being worked on
- Create the report(save to the database)
"""

#
reqcheq = RequirementAnalysis(user_req=request.requirement, report=json.dumps(report_dict),
                              atomicity_score=atomicity, measurability_score=measurability,
                              complexity_score=complexity, readability_score=readability, req_score=req_score )
session_gen = get_session()
session = next(session_gen)
db_save = create_report(reqcheq, session=session) # Save report to DB
session.add(db_save)
session.commit()

return {"reqcheq_id": db_save.id, # id to eventually pass to feedback endpoint
        "report": report_dict,
        "atomicity_score": atomicity,
        "measurability_score": measurability,
        "complexity_score": complexity,
        "readability_score": readability,
        "req_score": req_score}
```

For context, the schema for the database is shown below.

```python
# Table Model/Config: id, inputted user req, llm generated report, calculated req_score
class RequirementAnalysis(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    user_req: str = Field(index = True)
    report: str = Field(index = True)
    atomicity_score: int = Field(default=None, index = True)
    measurability_score: int = Field(default=None, index = True)
    complexity_score: int = Field(default=None, index = True)
    readability_score: int = Field(default=None, index = True)
    req_score: int = Field(default=None, index=True)
```

Notice the report, which in the analyzed endpoint was produced as a JSON formatted string and converted to a python dictionary. The reason for the python dictionary conversion is because in development, validating the output of the analyzed endpoint proved useful for ensuring data consistency. Although, pydantic(the validation python library) is not compatible with JSON formatted strings. Before saving the report to the DB, the report is converted back into JSON formatted string to match the data schema of our model.

Backend Implementation: Feedback Endpoint

The second endpoint is the feedback endpoint. The data validation only exists on the incoming data, which is an integer representing the id of the requirement analysis in order to access it in the database.

The feedback endpoint also leverages OpenAI's "gpt-4o-mini" model for a response. As opposed to generating a report or metrics, this endpoint is responsible for providing qualitative feedback on the inputted requirement. The first thing the endpoint does is read from the database, extracting the full requirement analysis, which includes the user requirement. The prompt instructs the LLM to take all the metrics of the requirement into consideration prior to providing feedback. This design is to prevent the LLM from providing feedback on the requirement outside the context of this project.

The LLM is instructed to carry out the following tasks:

- Begin with a short/minimal disclaimer that the feedback being provided is an AI generated analysis
- Provide general constructive feedback
- Provide feedback on what the requirement does well
- Provide feedback on what the requirement does poorly
- Suggest changes the user should consider making to the requirement
- Provide the user with an improved version of the originally inputted requirement

Again, the LLM is instructed and configured to provide the response in JSON formatted string.

```
- Generate/Output the following in JSON format. ONLY in JSON format as specified. Do not include explanations or extra text. Do not use code fences
1) Begin with a short/minimal disclaimer that this is an AI generated analysis
2) Provide general constructive feedback on the requirement, limit to 200 words
3) Explain what is good about the software requirement, limit to 200 words
4) Explain what was done poorly about the software requirements, limit to 200 words
5) Suggest changes the user should make to the requirement in order to fix it, limit to 200 words
6) Provide the user with an improved version of their originally inputted software requirement, limit to 200 words

OUTPUT FORMAT:
{{
    "disclaimer:" "AI Generated Feedback
    "feedback:" "provide general constructive feedback, limit to 200 words",
    "requirement_strengths": "speak about what the requirement does well, limit to 200 words",
    "requirement_weaknesses": "speak about what the requirement does poorly, limit to 200 words",
    "suggestions:" "walk over suggestions the user could make to improve their requirement, limit to 200 words",
    "improved_version": "Revise and rewrite an improved version of the software reuirement, keep it atomic",
}}

"""
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.0,
    response_format={"type": "json_object" }
)
```

Testing Procedures

Manual Testing:

LLM Responses:

The pitfall of relying on models to generate artifacts in development is they can make errors you can't predict. A core component of the development process was ensuring the LLMs were generating the correct output. The workflow consisted of writing the prompt, observing the generated response from the LLM for a variety of different cases, and refactoring the prompt. This process included multiple iterations, modifying the prompt where necessary. Because the LLM output is inherently non-deterministic, visual inspection and code review was the primary method for ensuring correct functionality.

Automated Testing:

The overall score has its own designated automated test. It's reliant on multiple components of the analyzed endpoint, including the non-deterministic behavior of the LLM responses. In order to ensure the overall test score functioned correctly, a simple test including a dummy report and defined scores ensures the overall scores equals the expected value.

```python
class TestOverallScore(unittest.TestCase):

    def test_overall_score(self):
        report = {
            "type": "Functional + Non-Functional",
            "functional_portion": "Valid",
            "non_functional_portion": "Valid",
            "class": "Functional",
            "ambiguity_score": 70,
            "active_voice": "Yes"
        }

        atomicity = 100
        measurability = 90
        complexity = 60
        readability = 40

        result = overall_score(
            report,
            atomicity,
            measurability,
            complexity,
            readability
        )

        # Expected breakdown:
        # type: +15
        # functional_portion: +5
        # non_functional_portion: +10
        # class: +5
        # ambiguity (70): +10
        # active_voice: +10
        # atomicity (100): +10
        # measurability (90): +10
        # complexity (>=50): +10
        # readability (>30): +10
        # TOTAL = 95

        self.assertEqual(result, 95)
```

Test Driven Development - 2 Cycles

Test 1:

Requirement: implement a working database capable of saving reports, or analysis of software requirements. The database must also be capable of getting a single report stored and all the reports stored in a database.

DB Model For Context:

```python
class RequirementAnalysis(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    user_req: str = Field(index = True)
    report: str = Field(index = True)
    req_score: int = Field(default=None, index=True)
```

Failing Test Code

→ Because my feature relies on LLM responses, it's inherently non-deterministic. Therefore, I've written dummy requirements and reports to save to the DB to ensure correct functionality.

```python
class TestDatabaseFunctions(unittest.TestCase):
    @unittest.expectedFailure
    def test_db_save(self):
        """
        Tests that my DB function correctly saves the requirement analysis to the database
        """

        # Dummy requirement to save in report
        requirement = "The system shall analyze the detected vulnerability. The system shall inspect SSL certificate. The system

        # Dummy report based on the requirement above
        report = {
            "report": {…
            "req_score": 90
                }
        db_entry = RequirementAnalysis(user_req = requirement, report = json.dumps(report), req_score=90) # Report must be string

        # Call DB Function
        session_gen = get_session() # Session = temporary workspace
        session = next(session_gen) # database.py generates sessions
        db_save = create_report(reqcheq=db_entry, session=session)
        session.commit()
        session.refresh(db_save)
        db_save_id = db_save.id

        # Unnittest assertions
        self.assertIsNotNone(db_save.id, "Should receive an id") # ensure the ID is valid
        self.assertEqual(db_save.id, db_save_id, "Report ID should match")
        self.assertEqual(db_save.user_req, requirement) # ensure the requirement matches the user requirement
        self.assertEqual(db_save.req_score, 90) # ensure the score is 90
        self.assertIsInstance(db_save.report, str) # ensure the report is a string
```

```python
@unittest.expectedFailure
def test_get_report(self):
    """
    Tests that the read_report DB function correctly retrieves a single report from the database
    """
    # Setup: Create and save a report first
    requirement = "The system shall log all user actions"
    report = {
        "report": {
            "functional_portion": "The system shall log all user actions",
            "non_functional_portion": "N/A",
            "ambiguity_level": "Low",
            "active_voice": "Yes"
        },
        "req_score": 95
    }
    db_entry = RequirementAnalysis(user_req=requirement, report=json.dumps(report), req_score=95)

    # Save the report
    session_gen = get_session()
    session = next(session_gen)
    saved_report = create_report(reqcheq=db_entry, session=session)
    session.add(saved_report)
    session.commit()
    session.refresh(saved_report)
    saved_id = saved_report.id

    # Call the read_report function
    saved_report = read_report(reqcheq_id=saved_id, session=session)

    # Assertions
    self.assertIsNotNone(saved_report, "Report should be found")
    self.assertEqual(saved_report.id, saved_id, "Report ID should match")
    self.assertEqual(saved_report.user_req, requirement, "Requirement should match")
    self.assertEqual(saved_report.req_score, 95, "Score should match")
```

```python
def test_get_all_reports(self):
    db_entry2 = RequirementAnalysis(user_req=requirement2, report=json.dumps(report2), req_score=95)

    requirement3 = "The system should store user information in the database securely"
    report3 = {
        "report": {…
        "req_score": 80
    }

    db_entry3 = RequirementAnalysis(user_req=requirement3, report=json.dumps(report3), req_score=95)

     # Save the reports
    session_gen = get_session()
    session = next(session_gen)

    # Delete all existing rows in table from previous tests
    session.exec(text("DELETE FROM requirementanalysis")) # table name is lowercase in SQL statement
    session.commit()

    saved_report1 = create_report(reqcheq=db_entry1, session=session)
    saved_report2 = create_report(reqcheq=db_entry2, session=session)
    saved_report3 = create_report(reqcheq=db_entry3, session=session)

    session.add(saved_report1)
    session.commit()
    session.add(saved_report2)
    session.commit()
    session.add(saved_report3)
    session.commit()
    session.refresh(saved_report1)
    session.refresh(saved_report2)
    session.refresh(saved_report3)

    # After saving and committing all reports, call read_reports to retrieve them:
    all_reports = read_all_reports(session=session) # session is only required param

    # Now you can add assertions on the returned reports:
    self.assertEqual(len(all_reports), 3, "Should return all 3 reports")

    # Assert each requirement committed above exists in DB
    self.assertIn(requirement1, [r.user_req for r in all_reports], "Report 1 should be in results")
    self.assertIn(requirement2, [r.user_req for r in all_reports], "Report 2 should be in results")
    self.assertIn(requirement3, [r.user_req for r in all_reports], "Report 3 should be in results")
```

Failing Test Output:

→ Notice the expected failure decorators before each test function. This removes any errors found from calling the function that isn't written yet and instead instructs the test case to expect an error. Since the database functions aren't written yet, the terminal will output different kinds of errors that aren't related to the logic of the function that's going to be written.

```
(venv) kondwani@KoNitro5:~/reqcheqai/backend$ python3 tests.
py
test_db_save (__main__.TestDatabaseFunctions.test_db_save)
Tests that my DB function correctly saves the requirement an
alysis to the database ... expected failure
test_get_all_reports (__main__.TestDatabaseFunctions.test_ge
t_all_reports)
Tests that the database functions successfully return all th
e reports in the DB ... expected failure
test_get_report (__main__.TestDatabaseFunctions.test_get_rep
ort)
Tests that the read_report DB function correctly retrieves a
 single report from the database ... expected failure

----------------------------------------------------------------
----------
Ran 3 tests in 0.025s

OK (expected failures=3)
```

Minimal Implementation

→ Docs Followed: https://fastapi.tiangolo.com/tutorial/sql-databases/?h=database#create-models

```
# Writes a requirement report to DB
def create_report(reqcheq: RequirementAnalysis, session: Session) ->
RequirementAnalysis:
     session.add(reqcheq)
     session.commit()
     session.refresh(reqcheq)
     return reqcheq

# Gets all the requirement reports in DB
def read_all_reports(
    session: Session,
    offset: int = 0, # number of reports to skip
    limit: int = 100, # Set limit to 100 reports returned
) -> list[RequirementAnalysis]:
    reqcheqs =
session.exec(select(RequirementAnalysis).offset(offset).limit(limit)).all(
) # offset and limits paginate results
    return reqcheqs
```

```
# Gets a single requirement report from DB
def read_report(reqcheq_id: int, session: Session) -> RequirementAnalysis:
    reqcheq = session.get(RequirementAnalysis, reqcheq_id)
    if not reqcheq:
        raise HTTPException(status_code=404, detail="Requirement Analysis
Not Found")
    return reqcheq
```

Passing Test Output

```
(venv) kondwani@KoNitro5:~/reqcheqai/backend$ python3 tests.
py
test_db_save (__main__.TestDatabaseFunctions.test_db_save)
Tests that my DB function correctly saves the requirement an
alysis to the database ... ok
test_get_all_reports (__main__.TestDatabaseFunctions.test_ge
t_all_reports)
Tests that the database functions successfully return all th
e reports in the DB ... ok
test_get_report (__main__.TestDatabaseFunctions.test_get_rep
ort)
Tests that the read_report DB function correctly retrieves a
 single report from the database ... ok

----------------------------------------------------------------
----------
Ran 3 tests in 0.033s

OK
```

Refactoring Notes

→ My test function for returning all reports was returning more reports than 3 because the previous tests also added reports to the DB. I needed to clear all rows in the DB before adding new rows to pass my test of 3 reports in the table. I used the below code before creating the reports to clear the DB(it is a SQL table, so it supports writing SQL statements):

```
# Delete all existing rows in table from previous tests
session.exec(text("DELETE FROM requirementanalysis")) # table name is
lowercase in SQL statement
        session.commit()
```

Test 2:

Requirement: Tests the computation of all 4 requirement scores: atomicity, measurability, complexity, and readability scores. Each score will be computed in its own unique way based on the contents of the requirement.

Failing Test Code:

```python
class TestScoreComputations(unittest.TestCase):
    def test_atomicity_score(self):
        # Case 1: Low atomicity requirement(only two verbs)
        req1 = "The system shall inspect SSL certificate"
        score1 = atomicity_score(req1)
        self.assertIsInstance(score1, int) # Should return an integer
        self.assertEqual(score1, 100)

        # Case 2: Poorly written requirement with high atomicity
        req2 = "The system shall receive input, send it to the backend, run it through the script, validate the output, and save to the database"
        score2 = atomicity_score(req2)
        self.assertLess(score2, 100)

    def test_measurability_score(self):
        # Case 1: Requirement written with measurable components
        req1 = "The system shall output the text to the user within 4 seconds of post request"
        score1 = measurability_score(req1)
        self.assertIsInstance(score1, int)
        self.assertEqual(score1, 100)

        # Case 2: Requirement written with poor measurability
        req2 = "The system shall run the script"
        score2 = measurability_score(req2)
        self.assertLess(score2, 50)

    def test_complexity_score(self):
        # Case 1: Well written requirement with low complexity
        req1 = "Upon registration, the system must generate a shared encryption key."
        score1 = complexity_score(req1)
        self.assertIsInstance(score1, int)
        self.assertEqual(score1, 100)

        # Case 2: Poorly Written requirement with high complexity
        req2 = "The system shall, in accordance with dynamically inferred user interaction patterns and contingent upon contextual session states " \
        "that may or may not persist across heterogeneous device environments, provide a mechanism through which end-users are enabled to initiate, modify, " 
        "and, where applicable, retroactively reconcile task-oriented data entities in a manner that is perceived as sufficiently intuitive, except in cases 
        "system-imposed constraints necessitate deviation from expected interaction paradigms, at which point alternative workflows should be made available 
        " significant cognitive overhead or degradation in overall performance characteristics."
        score2 = complexity_score(req2)
        self.assertLess(score2, 80)
```

```python
def test_readability_score(self):
    # Case 1: Well written requirement with high readability
    req1 = "The system shall output the text to the user within 4 seconds of post request"
    score1 = readability_score(req1)
    self.assertIsInstance(score1, int)
    self.assertGreaterEqual(score1,30)

    # Case 2:
    req2 = "The system shall, in accordance with dynamically inferred user interaction patterns and contingent upon contextual session states " \
    "that may or may not persist across heterogeneous device environments, provide a mechanism through which end-users are enabled to initiate, modify, " \
    "and, where applicable, retroactively reconcile task-oriented data entities in a manner that is perceived as sufficiently intuitive, except in cases where " \
    "system-imposed constraints necessitate deviation from expected interaction paradigms, at which point alternative workflows should be made available without introducing"
    " significant cognitive overhead or degradation in overall performance characteristics."
    score2 = readability_score(req2)
    self.assertIsInstance(score2, int)
    self.assertLess(score2, 30)
```

Failing Test Output:

```
(venv) kondwani@KoNitro5:~/reqcheqai/backend$ python3 tests.
py
test_atomicity_score (__main__.TestScoreComputations.test_at
omicity_score) ... expected failure
test_complexity_score (__main__.TestScoreComputations.test_c
omplexity_score) ... expected failure
test_measurability_score (__main__.TestScoreComputations.tes
t_measurability_score) ... expected failure
test_readability_score (__main__.TestScoreComputations.test_
readability_score) ... expected failure

----------------------------------------------------------------
----------
Ran 4 tests in 0.001s

OK (expected failures=4)
```

Minimal Implementation:

```
def atomicity_score(text: str):
    """
    Calculates an atomicity score. Good requirements describe one feature,
counting verbs informs
    how many features the requirement is specifying for implementation.
    """
    doc = nlp(text)

    # token = smallest analytical unit processed in
spacy(word,symbol,etc.)
    verbs = [token.lemma for token in doc if token.pos_ == "VERB"] #
Counts all the verbs in the requirement

    # Assigning an atomicity score based on number of verbs
    if len(verbs) == 0:
        return 0
    elif len(verbs) <= 4:
        return 100
    elif len(verbs) == 5:
        return 50
```

```
    elif len(verbs) >= 6:
        return 25

def measurability_score(text: str):
    """
    Computes a measurability score by searching through the requirement
and observing whether it contains
    a list of units, keywords, and/or spacy's like_num attribute. Spacy's
like_num finds descriptions of
    numbers(.e.g. "hundreds", "ten", etc.). Measurable requirements are
good requirements
    - like_nums get 40 points
    - units get 40 points
    - keywords get 20 points
    - returns a number that can't exceed 100
    """
    UNITS = {"seconds", "ms", "milliseconds", "%", "percent", "users",
"requests"} # common unit terms we may see in requirements
    KEYWORDS = {"within", "at least", "at most", "no more than"} #
keywords/phrases that imply measurement

    doc = nlp(text)

    score = 0

    # spacy like_num attribute
    if any(token.like_num for token in doc):
        score += 40

    if any(token.text.lower() in UNITS for token in doc):
        score += 40

    if any(keyword in text.lower() for keyword in KEYWORDS):
        score += 20

    return min(score, 100) # ensures score doesn't exceed 100, might fix
later

def complexity_score(request: UserRequirement):
    """
```

```
    Computes a complexity score of the requirement based on the sentence
length, the number of verbs, and the number of clauses.
    The more complex a sentence, the longer it is, the more verbs it
contains, the more clauses it contains. Making requirements
    precise and direct are important for implementation.
    """

    doc = nlp(request)
    clause_count = sum(1 for token in doc if token.dep_ in ["advcl",
"ccomp", "xcomp", "acl", "relcl"]) # Spacy labels that detect types of
cluases
    if clause_count <=2:
        return 100
    elif clause_count == 3:
        return 70
    elif clause_count == 4:
        return 50
    elif clause_count > 4:
        return 30



def readability_score(text: str):
    """
    Utilizes textstat's Flesch Reading Ease function. A formula that
calculates text readability based on sentence
    length and word length (syllables). Formula:
    206.835 - (1.015 x ASL) - (84.6 x ASW)
    ASL = Average Sentence Length (total words / total sentences).
    ASW = Average Syllables per Word (total syllables / total words). The
scoring system per docs:
    90-100 = Very Easy to read; 80-89 = Easy to read; 70-79 = fairly easy;
60-69 = standard; 50-59 = fairly difficult
    30-49 = difficult; 0-29 = confusing

    """
    readability_score = round(textstat.flesch_reading_ease(text))
    return readability_score
```

Passing Test:

```
(venv) kondwani@KoNitro5:~/reqcheqai/backend$ python3 tests.
py
test_atomicity_score (__main__.TestScoreComputations.test_at
omicity_score) ... ok
test_complexity_score (__main__.TestScoreComputations.test_c
omplexity_score) ... ok
test_measurability_score (__main__.TestScoreComputations.tes
t_measurability_score) ... ok
test_readability_score (__main__.TestScoreComputations.test_
readability_score) ... ok

----------------------------------------------------------------
----------
Ran 4 tests in 0.536s

OK
```

Refactoring Notes:

In writing out my methods that compute the score I discovered that the textstat function I intended on using assigns low scores to technically, higher education sentences. As a result, I readjusted the test case to take into consideration the low scores given to sentences, like the software requirements being written, being written in the context of our project. I modified my test case to consider anything higher than 30 to be a software requirement with high readability. Additionally, similar to the previous TDD cycle, I removed the decorator specifying to the system to expect a failed test case.

References

Aggarwal, Shivam Bansal, Chaitanya. "Textstat: Calculate Statistical Features from Text." *PyPI*, pypi.org/project/textstat/.

Ahmet Umut Alkan: Prompt & Flow. "FastAPI + LLM in under 10 Minutes." *YouTube*, 25 Aug. 2025, www.youtube.com/watch?v=dUCHfeUBC7A&list=PLcwCmgOOmhNVxn3Ai1rJzpXVTbtSGlnmM. Accessed 30 Apr. 2026.

Automate with Rakesh. "Python JSON Parsing: A Step-By-Step Guide to Extract Data from JSON." *YouTube*, 20 Aug. 2023, www.youtube.com/watch?v=nKtyWfqNmr8. Accessed 3 Aug. 2024.

"CORS (Cross-Origin Resource Sharing) - FastAPI." *Tiangolo.com*, 2025, fastapi.tiangolo.com/tutorial/cors/#use-corsmiddleware.

Dave Ebbelaar. "FastAPI for AI Projects - Getting Started in 15 Minutes." *YouTube*, 22 May 2025, www.youtube.com/watch?v=-IaCV5-mlSk&list=PLcwCmgOOmhNVxn3Ai1rJzpXVTbtSGlnmM&index=2. Accessed 30 Apr. 2026.

Eric Roby. "Python Unit Testing | FastAPI with Pytest Tutorial (Fast & Easy)." *YouTube*, 6 Oct. 2023, www.youtube.com/watch?v=jM-zWp8dNQA.

---. "Python Unit Testing | FastAPI with Pytest Tutorial (Fast & Easy)." *YouTube*, 6 Oct. 2023, www.youtube.com/watch?v=jM-zWp8dNQA. Accessed 18 Sept. 2025.

GeeksforGeeks. "NLP Libraries in Python." *GeeksforGeeks*, 2 Apr. 2024, www.geeksforgeeks.org/nlp/nlp-libraries-in-python/#2-nltk-natural-language-toolkit-library . Accessed 30 Apr. 2026.

Kadebu, Prudence. “Dataset of Students’ Software Projects Requirements (DOSSPRE).” *Mendeley Data*, vol. 2, Dec. 2024, https://doi.org/10.17632/23xtbvk6yp.2. Accessed 30 Apr. 2026.

“Linguistic Features · SpaCy Usage Documentation.” *Linguistic Features*, spacy.io/usage/linguistic-features.

pixegami. “FastAPI for AI: Build an AI Endpoint in 30 Minutes.” *YouTube*, 10 June 2025, www.youtube.com/watch?v=uDUfZyNXFX0&list=PLcwCmgOOmhNVxn3Ai1rJzpXVTbtSGlnmM&index=3. Accessed 30 Apr. 2026.

“SQL (Relational) Databases - FastAPI.” *Tiangolo.com*, 2026, fastapi.tiangolo.com/tutorial/sql-databases/?h=database#create-models. Accessed 30 Apr. 2026.